SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**
Question Count: 5

Code of Conduct

I _M.Mary Sharlin (192511330)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: M.Mary Sharlin____

Assessment Tasks
Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

HDFS-based storage solution for a media platform

Proposed design

Use an HDFS cluster with:

- **Replication factor = 3** — the maximum allowed.
- **NameNode High Availability (HA)** using an Active and Standby NameNode.
- **JournalNodes** for shared metadata/quorum.
- **DataNodes distributed across multiple racks** using rack awareness.
- Store media files as large HDFS blocks, e.g. **128–256 MB**, to reduce metadata overhead.
- Use **erasure coding for colder media**, where appropriate, to reduce storage overhead while maintaining fault tolerance.
- Separate **hot, warm, and cold media** using storage policies.

Design justification

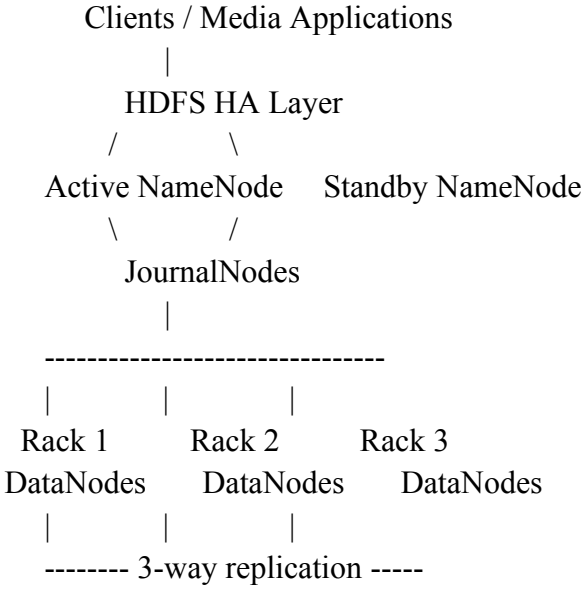
Availability > 99.9%:
NameNode HA prevents a single NameNode failure from making the filesystem unavailable. Replication across DataNodes and racks ensures that loss of a node or rack does not result in data loss.

Replication constraint:
The normal replication factor is exactly **3**, so it does not exceed the permitted maximum. Three replicas provide tolerance against multiple individual disk/node failures.

Cost efficiency:
Keeping all data at 3× replication can become expensive for a large media platform. Frequently accessed media can use 3-way replication, while older/cold data can use **HDFS Erasure Coding**, which provides fault tolerance with less storage overhead.

Scalability:
Additional DataNodes can be added as the media collection grows. HDFS automatically balances data across the cluster.

Architecture



Conclusion: This design provides >99.9% availability through HA and rack-aware replication while controlling storage cost through large blocks, tiering, and erasure coding for cold data.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

MapReduce workflow for processing logs within 20 minutes

Proposed job design

Use a **two-stage MapReduce workflow**:

1. **MapReduce Job 1 – Log parsing and aggregation**
2. **MapReduce Job 2 – Final aggregation/report generation**

If possible, combine the two stages into a single MapReduce job to reduce shuffle and startup overhead.

Mapper

The mapper reads log records and extracts useful fields such as:

timestamp
user_id
URL
HTTP status
response time
IP address

It emits key-value pairs such as:

(URL, 1)
(status_code, 1)
(user_id, 1)

Combiner

A **Combiner** performs local aggregation before data is transferred over the network.

For example:

Mapper output:
(URL_A, 1)
(URL_A, 1)
(URL_A, 1)

Combiner:
(URL_A, 3)

This significantly reduces shuffle traffic and is important when the cluster has limited nodes.

Reducer

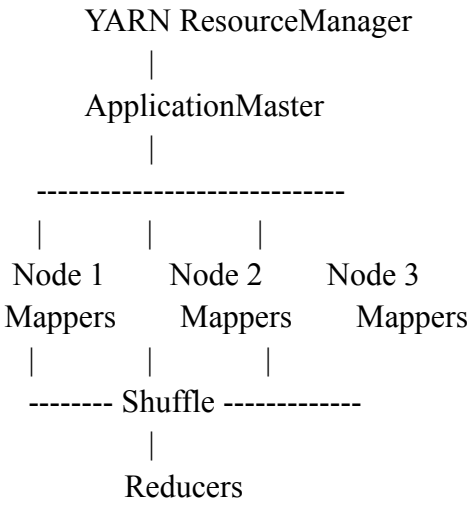
Reducers aggregate values:

(URL_A, 3)
(URL_B, 8)

and produce the final statistics.

Task distribution

Suppose there are only a few worker nodes:



The input logs are divided into **HDFS input splits**. Map tasks are scheduled close to the DataNodes containing the data (**data locality**) to reduce network traffic.

Reducers are distributed across available nodes according to cluster capacity.

Meeting the 20-minute constraint

Use:

- Large enough input splits to avoid excessive task startup overhead.
- **Combiners** to minimize shuffle traffic.
- Data locality.
- Appropriate mapper/reducer parallelism.
- Compression for intermediate map output.
- Avoid unnecessary MapReduce stages.
- Speculative execution only for genuinely slow/straggling tasks, because excessive speculation can waste limited resources.
- Partition data evenly to avoid reducer skew.

Example timing target

Input read + Map : 6 min
Shuffle + Sort : 5 min
Reduce : 5 min
Output/overhead : 2 min
Total : 18 min

This leaves approximately **2 minutes of safety margin** within the 20-minute batch window.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

YARN scheduler policy for ETL, reporting and ML

Proposed scheduler

Use the **Capacity Scheduler** with separate queues and resource guarantees.

```
root
|
+-- ETL         50%
|
+-- Reporting   25%
|
+-- ML          25%
```

The exact percentages can be adjusted according to workload requirements.

Queue	Priority	Resource allocation
ETL	Highest	50%
Reporting	Medium	25%
ML	Normal	25%

ETL receives priority because it is typically part of the organization's critical data pipeline.

Fairness

Although ETL has priority, reporting and ML must not be permanently starved.

Configure:

- Minimum resource guarantees for each queue.
- Queue/user limits.
- Maximum capacity limits.
- Fair allocation of resources within each queue.

For example, if ETL is idle, its unused capacity can temporarily be used by ML or reporting workloads.

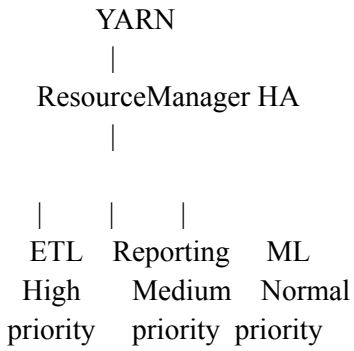
Node failure recovery

Use:

- **YARN ResourceManager HA** with Active and Standby ResourceManagers.
- NodeManagers send heartbeats to the ResourceManager.
- If a NodeManager stops responding, YARN marks the node unhealthy.
- Failed containers are rescheduled on healthy nodes.
- ApplicationMasters can recover/restart jobs as required.

- Persist intermediate/input data in HDFS rather than relying on local node storage.

Policy



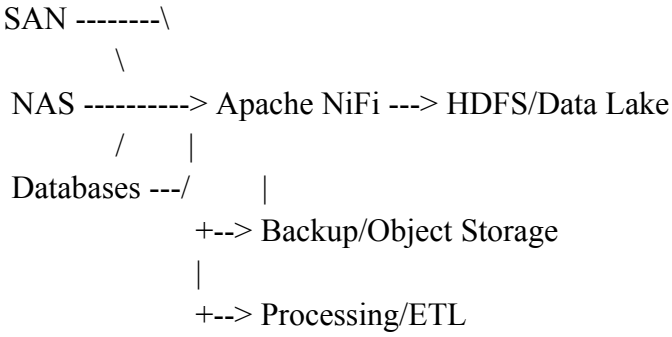
Result: ETL receives preferential access, while minimum resources and fair sharing protect reporting and ML. HA ResourceManagers and container rescheduling provide recovery from node failures.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

Enterprise ingestion from SAN, NAS and operational databases

Proposed architecture

Use **Apache NiFi** as the controlled ingestion layer, combined with HDFS/Hadoop storage.



1. SAN

SAN data can be exposed to the ingestion layer through appropriate mounted/file-system interfaces.

NiFi monitors and transfers files into HDFS.

2. NAS

NiFi can ingest files from NAS-mounted locations and move them into HDFS.

3. Operational databases

Use database connectors/JDBC-based ingestion.

For large databases, use:

- Incremental extraction.
- Timestamp/watermark columns.
- Change Data Capture (CDC), where available.

This avoids repeatedly copying the entire database.

Backup reliability

Use:

- HDFS replication for operational fault tolerance.
- A separate backup target such as enterprise object storage or another storage system.
- Scheduled backups/checkpoints.
- Checksums and validation.
- Versioned backup policies.

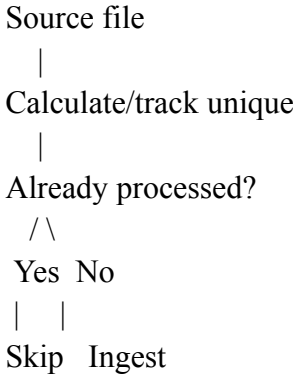
The backup system should be **independent of the primary HDFS cluster**, so that a cluster-wide failure does not destroy both the original and backup.

Minimal duplication

Use:

- Unique source identifiers.
- File hashes/checksums.
- Metadata tracking.
- Incremental database ingestion.
- NiFi state management.
- Idempotent ingestion pipelines.

For example:



Component	Purpose
NiFi	Data ingestion, routing, transformation and flow management
HDFS	Scalable primary data lake
JDBC/CDC	Incremental database ingestion
Metadata/catalog	Track source and lineage
Checksums	Detect duplicates/corruption

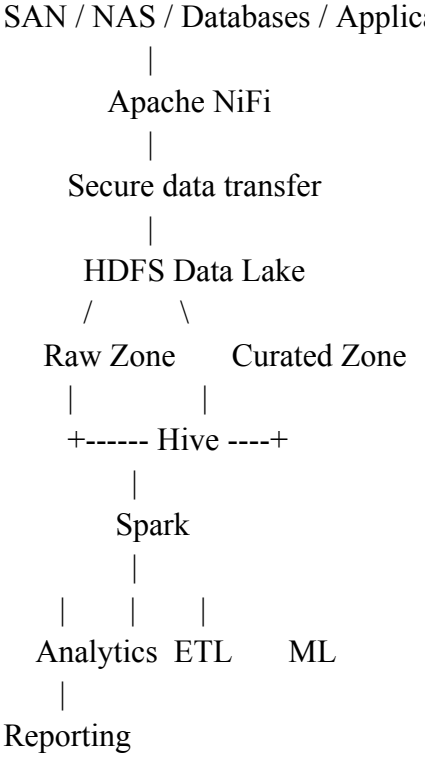
Component	Purpose
Backup storage	Disaster recovery
Hadoop/ETL	Large-scale processing

This architecture provides reliable ingestion from heterogeneous sources while minimizing repeated data movement and duplicate copies.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

End-to-end Hadoop ecosystem solution

Proposed architecture



Major components

1. Apache NiFi – ingestion

NiFi collects data from:

- Databases
- SAN/NAS
- Applications
- APIs

- Log sources

It provides flow management, routing, transformation and provenance.

2. HDFS – storage

HDFS provides scalable distributed storage.

Use:

- NameNode HA.
- Standby NameNode.
- JournalNodes.
- DataNode replication.
- Rack awareness.
- Appropriate storage policies.

This avoids a single point of failure.

3. YARN – resource management

YARN manages cluster resources between:

- ETL
- Analytics
- Reporting
- Machine learning

Use queue-based scheduling with ETL priority and fair resource sharing.

4. Hive – SQL analytics

Hive provides SQL-based access to data stored in HDFS and is suitable for large-scale analytical queries.

5. Spark – ETL and ML

Spark can process large datasets efficiently and supports:

- ETL
- Batch processing
- Machine learning
- Interactive analytics

Secure data movement

Use:

- **TLS/SSL** for data in transit.
- **Kerberos** for Hadoop authentication.
- **HDFS permissions/ACLs** for authorization.
- Encryption for sensitive data at rest.
- Secure NiFi connections.
- Service accounts with least-privilege access.

A simplified secure flow is:

Source
|
TLS / authenticated connection
|
NiFi
|
Kerberos + TLS
|
HDFS
|
YARN
/ | \
Hive Spark ML

Use HA for critical Hadoop control-plane components and distribute DataNodes across racks.

ETL integration

NiFi can perform initial ingestion and lightweight transformations, while Spark/Hive perform large-scale processing.

Example:

Database
↓
NiFi
↓
HDFS Raw Zone
↓
Spark ETL
↓
HDFS Curated Zone
↓
Hive
↓
Reports / BI / ML

Constraint	Solution
Secure data movement	TLS/SSL, Kerberos, authenticated NiFi connections
High availability	NameNode HA, standby services, replication, rack awareness
NiFi integration	NiFi handles ingestion, routing and provenance
ETL integration	Spark/Hive process data after ingestion
Scalability	Add DataNodes and YARN worker capacity

Constraint	Solution
Fault tolerance	HDFS replication and automatic task/container recovery
Security	Kerberos, ACLs, encryption and least privilege

Overall conclusion

The recommended ecosystem is **NiFi + HDFS + YARN + Hive + Spark**, with HA and security enabled throughout. NiFi provides reliable ingestion from heterogeneous enterprise sources, HDFS provides fault-tolerant scalable storage, YARN provides controlled resource sharing, and Hive/Spark provide ETL, analytics and ML capabilities. This satisfies the requirements for **secure data movement, high availability, and integration with ETL pipelines** while remaining scalable and manageable.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured